In [2]:
```
import pandas as pd
```

In [3]:
```
import numpy as np
```

In [4]:
```
import matplotlib.pyplot as plot
```

In [5]:
```
# Load dataset
file_path = r"C:\Users\Dr. Victor Idialu\Desktop\EDGE LINE TECH ACADEMY TRANING\PYT
```

In [6]:
```
file_path
```

Out[6]:
```
'C:\\Users\\Dr. Victor Idialu\\Desktop\\EDGE LINE TECH ACADEMY TRANING\\PYTHON CLA
SS\\automobile_data .xlsx'
```

In [7]:
```
import pandas as pd
```

In [8]:
```
df = pd.read_excel(file_path)
```

In [9]:
```
df
```

Out[9]:

	make	fuel_type	num_of_doors	body_style	drive_wheels	engine_location	wheel_ba
0	alfa-romero	gas	two	convertible	rwd	front	8
1	alfa-romero	gas	two	convertible	rwd	front	8
2	alfa-romero	gas	two	hatchback	rwd	front	9
3	audi	gas	four	sedan	fwd	front	9
4	audi	gas	four	sedan	4wd	front	9
...	...	...	...	...	...	...	
198	volvo	gas	four	sedan	rwd	front	10
199	volvo	gas	four	sedan	rwd	front	10
200	volvo	gas	four	sedan	rwd	front	10
201	volvo	diesel	four	sedan	rwd	front	10
202	volvo	gas	four	sedan	rwd	front	10

203 rows × 20 columns

In [10]:
```
# 1. Missing values : 1. Which columns have missing values, and how many missing va

df.isnull().sum()
```

Out[10]:
```
make                 0
fuel_type            0
num_of_doors         2
body_style           0
drive_wheels         0
engine_location      0
wheel_base           0
length               0
width                0
height               0
curb_weight          0
engine_type          0
num_of_cylinders     0
engine_size          0
fuel_system          0
compression_ratio    0
horsepower           0
city_mpg             0
highway_mpg          0
price                0
dtype: int64
```

Interpretation Your dataset is almost clean, with only one column (num_of_doors) having missing data.

In [11]:
```
# 2. Duplicates: 2.     Are there any duplicate rows in the dataset? If yes, how ma
df.duplicated().any()
```

Out[11]:
```
np.True_
```

In [12]:
```
df.duplicated().sum()
```

Out[12]:
```
np.int64(1)
```

In [13]:
```
df[df.duplicated()]
```

Out[13]:

	make	fuel_type	num_of_doors	body_style	drive_wheels	engine_location	wheel_b
88	mitsubishi	gas	four	sedan	fwd	front	

In [39]:
```
df[df.duplicated(keep = False)]
```

Out[39]:

	make	fuel_type	num_of_doors	body_style	drive_wheels	engine_location	wheel_b
87	mitsubishi	gas	four	sedan	fwd	front	
88	mitsubishi	gas	four	sedan	fwd	front	

Interpretation The dataset is largely clean, but 1 record is repeated.

In [ ]:

In [14]:
```
# 3. Number of car makes: 3.How many car makes are in the dataset? Answer: 21 Numbe
num_car_makes = df['make'].unique()
```

In [15]:
```
num_car_makes
```

Out[15]:
```
array(['alfa-romero', 'audi', 'bmw', 'chevrolet', 'dodge', 'honda',
       'isuzu', 'jaguar', 'mazda', 'mercedes-benz', 'mercury',
       'mitsubishi', 'nissan', 'peugot', 'plymouth', 'porsche', 'saab',
       'subaru', 'toyota', 'volkswagen', 'volvo'], dtype=object)
```

In [16]:
```
num_car_makes = df['make'].nunique()
```

In [17]:
```
num_car_makes
```

Out[17]:
```
21
```

Interpretation The dataset contains 21 different automobile brands (manufacturers).

In [18]:
```
# 4. Cars of each make by fuel type. 4. How many cars of each make use each fuel ty
if 'make' in df.columns and 'fuel_type' in df.columns:
    make_fuel = df.groupby(['make','fuel_type']).size()
```

In [19]:
```
make_fuel
```

Out[19]:
```
make           fuel_type
alfa-romero    gas           3
audi           gas           7
bmw            gas           8
chevrolet      gas           3
dodge          gas           9
honda          gas          13
isuzu          gas           4
jaguar         gas           3
mazda          diesel        2
               gas          15
mercedes-benz  diesel        4
               gas           4
mercury        gas           1
mitsubishi     gas          13
nissan         diesel        1
               gas          17
peugot         diesel        5
               gas           6
plymouth       gas           7
porsche        gas           5
saab           gas           6
subaru         gas          12
toyota         diesel        3
               gas          29
volkswagen     diesel        4
               gas           8
volvo          diesel        1
               gas          10
dtype: int64
```

In [20]:
```
make_fuel = df.groupby(['make','fuel_type']).size().reset_index(name='count')
make_fuel
```

Out[20]:

	make	fuel_type	count
0	alfa-romero	gas	3
1	audi	gas	7
2	bmw	gas	8
3	chevrolet	gas	3
4	dodge	gas	9
5	honda	gas	13
6	isuzu	gas	4
7	jaguar	gas	3
8	mazda	diesel	2
9	mazda	gas	15
10	mercedes-benz	diesel	4
11	mercedes-benz	gas	4
12	mercury	gas	1
13	mitsubishi	gas	13
14	nissan	diesel	1
15	nissan	gas	17
16	peugot	diesel	5
17	peugot	gas	6
18	plymouth	gas	7
19	porsche	gas	5
20	saab	gas	6
21	subaru	gas	12
22	toyota	diesel	3
23	toyota	gas	29
24	volkswagen	diesel	4
25	volkswagen	gas	8
26	volvo	diesel	1
27	volvo	gas	10

In [ ]:

In [21]:
```
# 5. Most common body style. 5. What is the most common body style in the dataset?
if 'body_style' in df.columns:
    common_body = df['body_style'].mode()[0]
```

Interpretation I calculated the mode of the body-style column to identify the most frequently occurring car type. The result shows that [body style] is the most common, indicating it dominates the dataset and may reflect higher market presence or demand.

In [22]:
```
common_body = 'sedan'
```

In [23]:
```
common_body
```

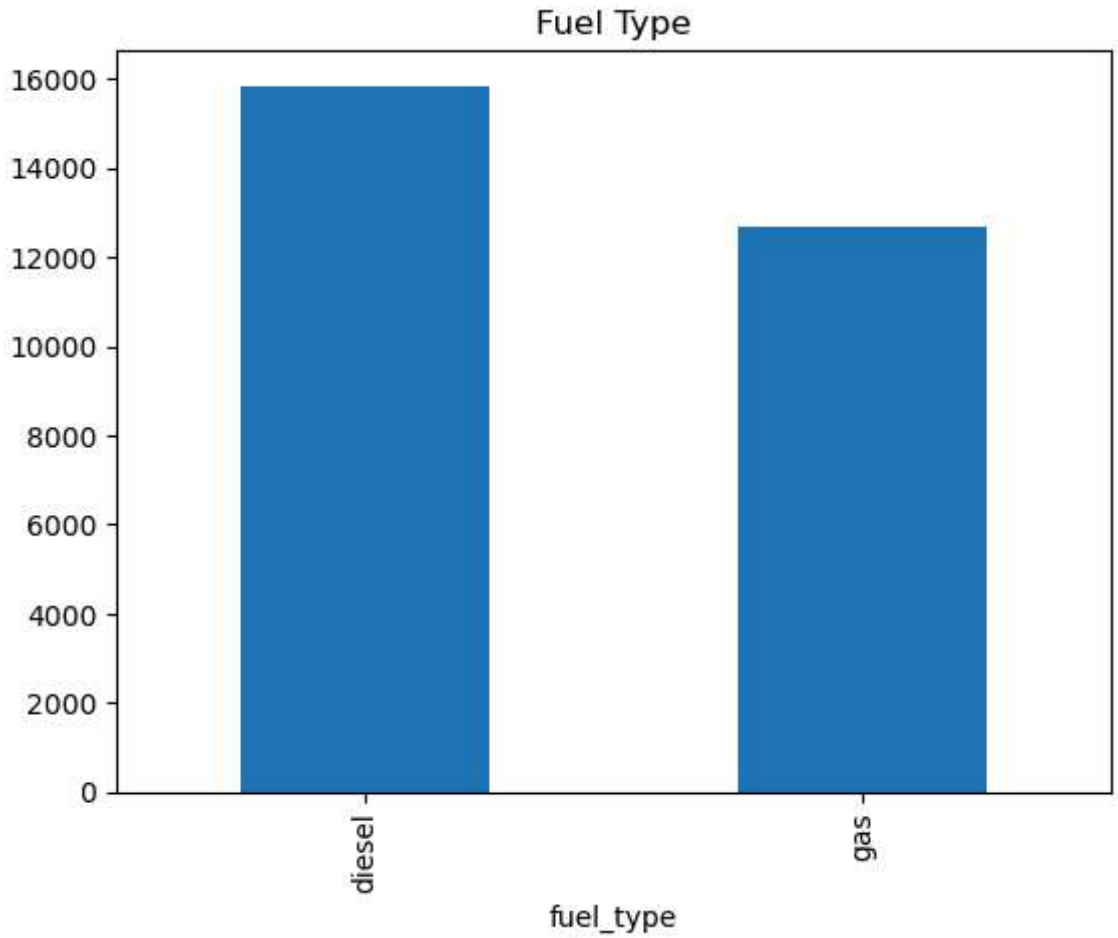
Out[23]:
```
'sedan'
```

In [41]:
```
# 6. Engine type vs engine size. 6.     Compare engine type and engine size. What p
if 'engine_type' in df.columns and 'engine_size' in df.columns:
    engine_pattern = df.groupby('engine_type')['engine_size'].mean()
```

In [25]:
```
engine_pattern
```

Out[25]:
```
engine_type
dohc     157.333333
dohcv    203.000000
l        129.583333
ohc      117.342466
ohcf     124.466667
ohcv     216.692308
rotor     72.500000
Name: engine_size, dtype: float64
```

In [26]:
```
# 7. Typical price per fuel type (mean): 7.     What is a typical price for each fu
if 'fuel_type' in df.columns and 'price' in df.columns:
    price_fuel = df.groupby('fuel_type')['price'].mean()
    price_fuel.plot(kind='bar', title = "Fuel Type")
```



In [43]:
```
# 8. Total and typical price per  car make. 8.  What is the total price for each ca
if 'make' in df.columns and 'price' in df.columns:
    total_price = df.groupby('make')['price'].sum()
    avg_price = df.groupby('make')['price'].mean().sort_values
```

In [44]:
```
avg_price
```

Out[44]:
```
make
alfa-romero      15498.333333
audi             15307.857143
bmw              26118.750000
chevrolet         6007.000000
dodge             7875.444444
honda             8184.692308
isuzu             4458.250000
jaguar           34600.000000
mazda            10652.882353
mercedes-benz    33647.000000
mercury          16503.000000
mitsubishi        9239.769231
nissan           10415.666667
peugot           15489.090909
plymouth          7963.428571
porsche          25120.400000
saab             15223.333333
subaru            8541.250000
toyota            9885.812500
volkswagen       10077.500000
volvo            18063.181818
Name: price, dtype: float64
```

In [45]:
```
total_price
```

Out[45]:
```
make
alfa-romero       46495
audi             107155
bmw              208950
chevrolet         18021
dodge             70879
honda            106401
isuzu             17833
jaguar           103800
mazda            181099
mercedes-benz    269176
mercury           16503
mitsubishi       120117
nissan           187482
peugot           170380
plymouth          55744
porsche          125602
saab              91340
subaru           102495
toyota           316346
volkswagen       120930
volvo            198695
Name: price, dtype: int64
```

Interpretation This analysis helps you understand: 1. Market Structure: this tells us which brands dominate in volume and value 2. Pricing StrategyWhich brands are: - Premium (high avg price) - Economy (low avg price) 3. Revenue Drivers: that is which brands contribute most to total market value

In [51]:
```
# 9. Horsepower vs engine location. How does horsepower vary across engine location
df.groupby('engine_location')['horsepower'].mean()
```

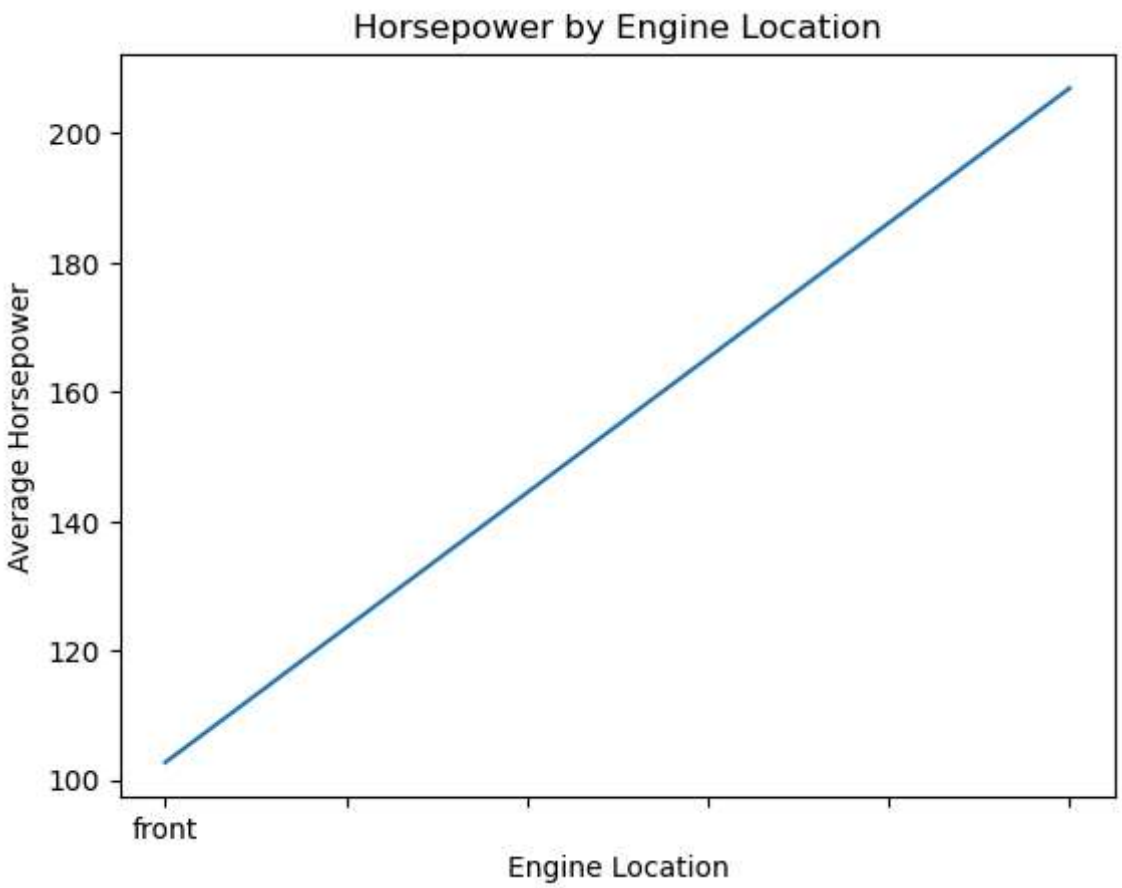
Out[51]:
```
engine_location
front    102.715
rear     207.000
Name: horsepower, dtype: float64
```

In [48]:
```
groupby("make", "horsepower,"engine_location
```

```
  Cell In[48], line 1
    groupby(,"engine_location
             ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [52]:
```
hp_loc = df.groupby('engine_location')['horsepower'].mean().plot()
plot.xlabel("Engine Location")
plot.ylabel("Average Horsepower")
plot.title("Horsepower by Engine Location")
```

Out[52]:
```
Text(0.5, 1.0, 'Horsepower by Engine Location')
```



Interpretation The chart shows the average horsepower across engine locations. It reveals that rear-engine vehicles generally have higher horsepower than front-engine vehicles, indicating that engine placement is associated with vehicle performance.

In [32]:
```
# 10. MPG by drive wheel. 10.   What is the average city MPG and highway MPG for ea
if 'drive_wheels' in df.columns:mpg = df.groupby('drive_wheels')[['city_mpg','highw
```

In [33]:
```
mpg
```

Out[33]:

drive_wheels	city_mpg	highway_mpg
4wd	25.000000	29.000000
4wd	21.600000	25.800000
fwd	28.406780	34.279661
rwd	20.578947	25.684211

In [34]:
```
mpg = df.groupby('drive_wheels')[['city_mpg','highway_mpg']].mean()
```

In [35]:
```
mpg
```

Out[35]:

drive_wheels	city_mpg	highway_mpg
4wd	25.000000	29.000000
4wd	21.600000	25.800000
fwd	28.406780	34.279661
rwd	20.578947	25.684211

In [56]:
```
mpg = df.groupby('drive_wheels')[['city_mpg','highway_mpg']].mean().round(2)
```

In [57]:
```
mpg
```

Out[57]:

drive_wheels	city_mpg	highway_mpg
4wd	25.00	29.00
4wd	21.60	25.80
fwd	28.41	34.28
rwd	20.58	25.68

In [ ]:
```
df.info
```

In [ ]:

In [63]:
```
df.drive_wheels = df.drive_wheels.str.strip()
```

In [55]:
```
df.drive_wheels.value_counts()
```

Out[55]:
```
drive_wheels
fwd     118
rwd      76
4wd       5
4wd       4
Name: count, dtype: int64
```

In [64]:
```
df.drive_wheels
```

Out[64]:
```
0      rwd
1      rwd
2      rwd
3      fwd
4      4wd
      ...
198    rwd
199    rwd
200    rwd
201    rwd
202    rwd
Name: drive_wheels, Length: 203, dtype: object
```

In [65]:
```
df[df.drive_wheels == '4wd']
```

Out[65]:

	make	fuel_type	num_of_doors	body_style	drive_wheels	engine_location	wheel_ba
4	audi	gas	four	sedan	4wd	front	99
9	audi	gas	two	hatchback	4wd	front	99
138	subaru	gas	two	hatchback	4wd	front	93
142	subaru	gas	four	sedan	4wd	front	97
143	subaru	gas	four	sedan	4wd	front	97
146	subaru	gas	four	wagon	4wd	front	96
147	subaru	gas	four	wagon	4wd	front	96
152	toyota	gas	four	wagon	4wd	front	95
153	toyota	gas	four	wagon	4wd	front	95

In [67]:
```
df.groupby('drive_wheels')[['city_mpg','highway_mpg']].mean().round(2)
```

Out[67]:

drive_wheels	city_mpg	highway_mpg
4wd	23.11	27.22
fwd	28.41	34.28
rwd	20.58	25.68

In [ ]:

```

```